

CloudGuardian

Final Capstone Report - Outline & Authoring Template

IIT Roorkee - PG Certificate in AI Powered Cybersecurity

Track: Microsoft Azure

Author: Kedar Pavaskar | Prepared as of: Week 3 submission

How to use this outline

This is a fill-in template, not the final report. Every gray italic line is authoring guidance - delete it once you've written the real content in its place. Headings are already styled so the Table of Contents below updates automatically (right-click it in Word > Update Field). Target length: 12-15 pages including the appendix, per the capstone brief.

1. Executive Summary

~1 page. Write this LAST, even though it's read first. State: what CloudGuardian is, which cloud (Azure) and why, the 3-week arc (Build & Break -> Detect & Prioritize -> Remediate & Govern), and your single strongest result (e.g. "16 deliberate misconfigurations across 5 categories, detected by 3 independent CSPM tools, with 6 controls under fully automated, human-gated, audited remediation").

2. Project Objectives & Scope

State the IIT Roorkee capstone brief's objectives verbatim, then your specific scope decisions: individual track (15-20 misconfigurations), Azure-only (teammate covers AWS), which 6 controls you chose for automated remediation and why those 6 specifically.

- Deploy a secure, realistic 3-tier Azure workload via Terraform
- Establish a clean security baseline, then introduce controlled misconfigurations
- Detect findings with multiple independent CSPM tools and cross-reference results
- Prioritize findings with a defensible, formula-based risk score
- Use an LLM to explain findings in plain English, with a verification step against raw scanner data
- Automate safe remediation for a defined subset of controls, gated by human approval for high-risk changes
- Continuously govern - detect drift after remediation, not just fix once
- Map every control to ISO 27001 Annex A, Microsoft Cloud Security Benchmark, and DPDP Act 2023

3. Architecture Overview

Insert your end-to-end architecture diagram here (Week 1 infra + Week 3 remediation plane in one picture). Walk through it left to right: workload -> CSPM scanners -> findings DB -> Event Grid -> [auto path / approval path] -> Function App -> target resources -> Log Analytics (audit trail) -> Automation Account (drift check). Reference the individual week diagrams from your Week 1 and Week 2 submissions if you produced them separately.

4. Week 1 Recap - Build and Break

4.1 Infrastructure

Summarize the Terraform stack: Resource Group, VNet + 2 subnets + NSG, Linux VM web tier, Azure SQL Database, Storage Account, Key Vault, Log Analytics Workspace. State the region and note any Azure platform constraints you discovered (e.g. TDE is a database attribute not a separate resource; minimum TLS below 1.2 is retired on SQL servers) - these are genuine engineering findings, include them, they show depth.

4.2 Misconfiguration Catalogue

Insert or reference your full misconfiguration catalogue (16 toggles across IAM, storage, networking, encryption, logging). One row per misconfig: description, purpose, expected CSPM detection, severity, MITRE ATT&CK mapping, CIS/ISO mapping, how to revert.

4.3 Baseline Scanning

Report your 3 scanner results side by side (Prowler / ScoutSuite / Steampipe): finding counts, PASS/FAIL breakdown, and where the tools agreed vs. disagreed. Note ScoutSuite's maintenance status as a caveat on its reliability as a cross-check.

5. Week 2 Recap - Detect and Prioritize

5.1 Unified Findings Database

Describe your normalized schema (the same one your dashboard's `data_loader.py` uses) and why a single canonical schema across tools/clouds matters.

5.2 Prioritization Model

State your formula (e.g. $CVSS \times Exposure \times Blast\ Radius \rightarrow Risk\ Score$), explain every term, and show 2-3 worked examples.

5.3 LLM Verification Methodology

Describe how you checked the LLM's plain-English explanations against raw scanner data, and how you flagged hallucinations. This connects directly to Week 3's privacy-preserving pipeline (Section 6.5) - the SAME verification discipline is applied there, extended with pseudonymization.

6. Week 3 - Remediate and Govern

6.1 Remediation Engine Architecture

The remediation plane is event-driven, not polling-based. Every finding is published once to an Event Grid Custom Topic; two Event Grid subscriptions route on severity alone - Low/Medium go straight to an Azure Function (auto-remediate), High/Critical go to a Logic App that gates on human approval before calling the same Function. Both paths converge on one Python dispatcher (`shared/remediation_engine.py`) so remediation LOGIC exists in exactly one place regardless of how it was authorized.

Insert your own sequence diagram or a screenshot of the Terraform apply output here. State WHY you chose severity as the routing key rather than e.g. resource type.

6.2 The Six Safe Remediation Functions

Control ID	What it fixes	CIS / MCSB reference
storage_public_access	Disables account-level anonymous blob public access	CIS 3.6; MCSB DP-2/NS-2
storage_encryption	Enforces HTTPS-only + TLS 1.2 minimum	CIS 3.1/3.12; MCSB DP-3/DP-4
diagnostic_logging	Re-attaches diagnostic settings to Log Analytics	CIS 5.1.x; MCSB LT-3/LT-4
sql_encryption	Enables Transparent Data Encryption	CIS 4.1.1; MCSB DP-4
keyvault_firewall	Sets network ACL default action to Deny	MCSB NS-2/DP-8
tagging	Merges required governance tags	MCSB GS-1

For each row, cite the finding ID it remediates, show a before/after Portal screenshot, and the audit log entry.

6.3 Human Approval Workflow

High/Critical findings never execute automatically. Event Grid calls a thin Logic App, which forwards the finding to the Function App; the Function App builds and sends an Approve/Reject email via Azure Communication Services (authenticated with its own Managed Identity - no mailbox, no OAuth) summarizing the finding, severity, resource, and proposed fix. Clicking Approve hits the Function's `approval_decision` endpoint directly, which validates a shared secret carried in the link before executing

the same remediation dispatcher; clicking Reject - or no response at all - leaves the resource untouched. Every decision is written to the audit log (Section 6.4's Log Analytics trail), independent of the email provider.

Screenshot the actual approval email and the Log Analytics query showing the resulting approval_decision audit record. Note for your defense: this design deliberately avoids depending on a Microsoft 365 / Exchange Online mailbox - an earlier version used the Office 365 Outlook connector's native Approve/Reject Actionable Message, which needs an M365 tenant most lab environments don't have. The one documented trade-off is that a GET-triggered approval link can, in principle, be pre-fetched by an email security scanner before a human clicks it; the shared-secret check mitigates unauthorized use, and the documented future-work fix is a two-step confirmation page (see functions/shared/notifications.py's docstring).

6.4 Governance - Continuous Drift Detection

A daily Azure Automation runbook (Test-RemediationDrift.ps1) re-checks all 6 controls' live state using the same read-only Az PowerShell calls a human auditor would use, and raises a Warning - not a silent re-fix - if any control has drifted back to non-compliant. This distinguishes CloudGuardian from a one-time fix-and-forget script: remediation without ongoing verification is not governance.

6.5 Privacy-Preserving LLM Pipeline

Before any finding is sent to an LLM for a plain-English explanation, structured fields (subscription ID, object ID, UPN, resource name, principal name) are replaced with deterministic tokens. A hard guardrail (verify_no_leakage) re-scans the exact outbound payload for GUID/email/resource-name patterns and BLOCKS the call if anything slipped through - it does not just warn. Detokenization happens only after the response returns, inside the Azure boundary. This directly implements the 'encryption, obfuscation, masking, or virtual tokens' safeguard named in the DPDP Rules, 2025.

Include the tokenize -> verify -> call -> detokenize sequence diagram, and a redacted example: real finding -> tokenized payload -> LLM response -> restored explanation.

7. Compliance Mapping

Every Week 3 control is mapped to ISO/IEC 27001:2022 Annex A, the Microsoft Cloud Security Benchmark v1, and the DPDP Act 2023 + DPDP Rules 2025 (notified 14 November 2025). See the full crosswalk workbook (Week3_Compliance_Crosswalk.xlsx) referenced in Appendix B.

Summarize the headline numbers here: how many Annex A controls touched, which MCSB domains, and the specific DPDP Rules 2025 safeguard your privacy pipeline implements. State the DPDP Act's phased implementation timeline honestly - main compliance duties become mandatory 13 May 2027 - and frame your work as 'ahead of the mandatory date' rather than implying it's currently a legal requirement.

8. Testing & Validation Evidence

This is your proof section. For at least 2 of the 6 controls, walk through: (1) misconfig toggled on via Terraform, (2) CSPM tool flags it, (3) finding published to Event Grid, (4) [approval email + click, if High/Critical], (5) Function executes, (6) Log Analytics audit entry, (7) CSPM re-scan shows it cleared. Screenshots at every step. This maps directly to the 'killer demo' you'll run live for your defense (see Week3_Demo_Script.docx).

9. Challenges & Lessons Learned

Be specific and technical - evaluators value real engineering friction over a frictionless narrative. Candidates from your own project history: Prowler hitting Windows MAX_PATH limits; Azure resource types that don't exist as

expected (TDE); retired platform settings (TLS<1.2); discovering mid-build that the Office 365 Outlook connector approach for the approval email required a Microsoft 365 mailbox unavailable in this lab environment, and redesigning around Azure Communication Services Email + Managed Identity instead (which also removed the azapi provider dependency the earlier Logic-App-JSON design needed); PowerShell COM [ref] marshalling; why keyvault_firewall is approval-gated even at low severity.

10. Security Best Practices Applied

- Zero Trust - no static secrets; Function App and Automation Account authenticate via System-Assigned Managed Identity only
- Least privilege - a custom RBAC role scoped to only the actions the 6 remediations need, not built-in Contributor
- Defense in depth - Key Vault firewall change is approval-gated specifically because it's the one control that could break a legitimate caller
- Auditability - every remediation, approval, and drift check is independently logged and queryable via KQL
- Privacy by design - LLM calls are tokenized and leakage-guarded before any data leaves the Azure boundary
- Reversibility - every remediation is a documented, idempotent, single-property change, not a destructive operation

11. Conclusion & Future Work

Restate your strongest result. Then list concrete next steps: wire the existing Streamlit dashboard to read Function/Logic App execution status live; extend the drift runbook to auto-open a ticket; add the remaining ~10 misconfigurations from your Week 1 catalogue to the remediation registry; move Key Vault access from function keys to full Azure AD Easy Auth.

12. References

Cite Microsoft Learn docs for every service used, the ISO 27001:2022 standard, the MCSB documentation, and the DPDP Act 2023 / Rules 2025 official notifications (PIB press release / Gazette). Use full URLs.

Appendix A - Terraform Resource Inventory

Paste `terraform state list` output from both Week 1 and Week 3 stacks here.

Appendix B - Compliance Crosswalk

Reference: Week3_Compliance_Crosswalk.xlsx (attached separately). Optionally paste the table as an image here for a self-contained PDF export.

Appendix C - Code Listing Index

List every file in the week3/ folder with a one-line description - effectively a repo map. Point to the GitHub repo URL rather than pasting full source into the report.